

What is Recursion?

Recursion is a programming technique where a method **calls itself** to solve a problem by breaking it down into smaller, identical sub-problems.

The Halting Condition

This is the most critical part of recursion. Without a **base case** to stop the method from calling itself, the program will crash with a `StackOverflowError`.

```
public static int sum(int k) {  
    if (k > 0) {  
        return k + sum(k - 1); // Recursive Call  
    } else {  
        return 0; // Halting Condition (Base Case)  
    }  
}
```

Recursion vs. Iteration

- Memory

Feature	Recursion	Loops (Iteration)
Mechanism	Method calls itself	Repeats a block (for/while)
Complexity	Ideal for trees/sorting	Best for simple sequences
Uses Stack Memory	Uses less memory	

Advanced Example: Factorials

Factorial logic (e.g., $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$) is a classic use case for recursion in mathematical programming.

```
static int factorial(int n) {  
    if (n > 1) {  
        return n * factorial(n - 1);  
    }  
    return 1;  
}
```

Common Recursive Errors

⚠ **StackOverflowError:** This happens when the recursion depth is too high or the base case is never reached.

Bad Example: `static void run(int n) { run(n); }` — This has no exit path.

Quick Summary

- Recursion translates complex math definitions into clean code.
- Always ensure every recursive step moves closer to the **Halting Condition**.
- Use loops instead of recursion for very high-repetition tasks to save memory.